

deepseekV4技术点分析



DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence

DeepSeek-AI
research@deepseek.com

Abstract

We present a preview version of DeepSeek-V4 series, including two strong Mixture-of-Experts (MoE) language models — DeepSeek-V4-Pro with 1.6T parameters (49B activated) and DeepSeek-V4-Flash with 284B parameters (13B activated) — both supporting a context length of one million tokens. DeepSeek-V4 series incorporate several key upgrades in architecture and optimization: (1) a hybrid attention architecture that combines Compressed Sparse Attention (CSA) and Heavily Compressed Attention (HCA) to improve long-context efficiency; (2) Manifold-Constrained Hyper-Connections (*m*HC) that enhance conventional residual connections; (3) and the Muon optimizer for faster convergence and greater training stability. We pre-train both models on more than 32T diverse and high-quality tokens, followed by a comprehensive post-training pipeline that unlocks and further enhances their capabilities. DeepSeek-V4-Pro-Max, the maximum reasoning effort mode of DeepSeek-V4-Pro, redefines the state-of-the-art for open models, outperforming its predecessors in core tasks. Meanwhile, DeepSeek-V4 series are highly efficient in long-context scenarios. In the one-million-token context setting, DeepSeek-V4-Pro requires only 27% of single-token inference FLOPs and 10% of KV cache compared with DeepSeek-V3.2. This enables us to routinely support one-million-token contexts, thereby making long-horizon tasks and further test-time scaling more feasible. The model checkpoints are available at <https://huggingface.co/collections/deepseek-ai/deepseek-v4>

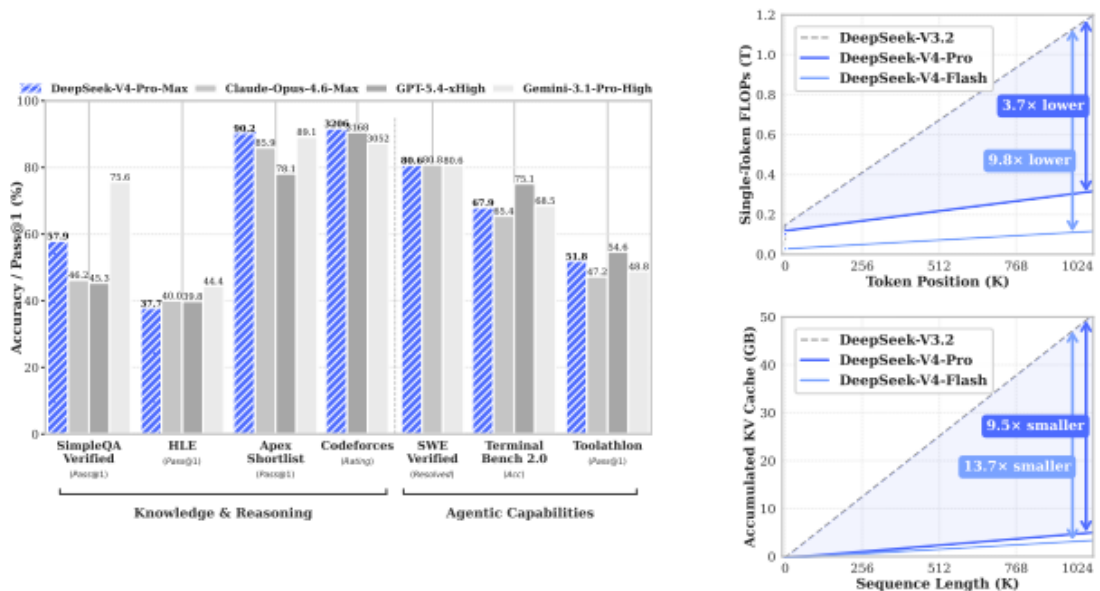


Figure 1 | Left: benchmark performance of DeepSeek-V4-Pro-Max and its counterparts. Right: inference FLOPs and KV cache size of DeepSeek-V4 series and DeepSeek-V3.2.

主要的技术点：

- Hybrid Attention with CSA and HCA (注意力，大幅提升长上下文效率)
- Manifold-Constrained Hyper-Connections (mHC) (流形约束超连接，替代传统残差连接，增强信号传播)
- Muon Optimizer (优化器，更快的收敛速度和更大的训练稳定性)
- FP4 Quantization-Aware Training
- Pre-Training: Anticipatory Routing and SwiGLU Clamping
- Post-Training: Specialist Training and On-Policy Distillation
- Reasoning Modes

一、流形约束超连接（Manifold-Constrained Hyper-Connections）

DeepSeek-V4 系列引入了流形约束超连接（mHC）（Xie 等人，2026），以加强相邻 Transformer 块之间的传统残差连接。与朴素超连接（HC）（Zhu 等人，2025）相比，mHC 的核心思想是将残差映射约束在特定的流形上，从而在保持模型表达能力的同时，增强跨层信号传播的稳定性。本小节简要介绍标准 HC，并描述我们如何设计 mHC 以实现稳定训练。

特性	残差连接 (Residual)	超连接 (HC)	流形约束超连接 (mHC)
核心公式	$X_{l+1} = X_l + F_l(X_l)$	$X_{l+1} = B_l X_l + C_l F_l(A_l X_l)$	$X_{l+1} = \bar{B}_l X_l + C_l F_l(A_l X_l)$
关键操作 拆解	恒等连接 直接将输入 X_l 加到输出上	可学习连接矩阵 A_l: 预处理输入，准备数据 B_l: 残差通路矩阵，融合历史特征 C_l: 后处理映射，将新特征写回流	约束连接矩阵 与HC结构一致，但对 B_l 施加“双随机矩阵”约束 $\bar{B}_l = \text{Sinkhorn-Knopp}(\tilde{B}_l)$
维度变化	维度不变	流宽度扩展	流宽度扩展与HC一致 保持 $n \times C$ 的宽残差流

	要求输入输出的特征维度 C 严格相等	引入超参数 n ，将隐含流宽度从 C 扩展到 $n \times C$	
参数性质	无参数 直接相加，不引入额外可学习参数	全参数化 $A/B/C$ 均为可学习的矩阵，其中 B 为 $n \times n$ 矩阵	约束参数化与HC类似 但强制 B 必须符合“双随机矩阵”的数学性质
核心特点	· 结构极简，保障梯度畅通 · 恒等映射，深层仍可访问浅层信息	· 拓宽信息高速路：不再局限于单车道，增加网络容量 · 动态路由：网络自己学习如何组合新旧信息	· 数学约束 + 硬件优化 · 保留HC的宽连接优势，解决其深层堆叠后的不稳定性

HC核心公式

1. 核心公式

论文中给出的朴素超连接 (HC) 状态更新公式如下：

$$X_{l+1} = B_l X_l + C_l F_l (A_l X_l)$$

公式中各符号的物理意义

符号	含义	维度 (Shape)	作用
X_l	第 l 层前的残差状态	$n_{hc} \times d$	宽带信息流，存储了扩展后的上下文信息
A_l	输入映射 (Input Mapping)	$1 \times n_{hc}$	“变细”：将宽状态 X_l 投影回 d 维，以便喂给计算层 F_l
F_l	实际计算层 (如 MoE)	$d \rightarrow d$	核心计算单元，处理实际的 d 维数据
C_l	输出映射 (Output Mapping)	$n_{hc} \times 1$	“变宽”：将计算层输出的 d 维结果，扩展回 $n_{hc} \times d$ 形状，注入残差流
B_l	残差变换 (Transformation)	$n_{hc} \times n_{hc}$	在宽状态内部进行线性变换（在朴素 HC 中通常是自由参数，但在 mHC 中会被约束）

2. 直观理解

你可以把这个过程想象成 “高速公路收费站”：

1. AI (汇聚)：多车道 ($n_{hc} \times d$) 的车流汇聚成单车道 (d)，进入收费站（计算层 F_l ）。
2. FI (处理)：单车道进行收费处理（计算）。

3. Cl (分流)：处理完的单车道再次分流，重新变回多车道 (nhc×d) 继续行驶。

朴素超连接 (HC) 的问题在于，矩阵 Bl 如果没有限制，这个多车道的信号在层层传递中容易发生数值震荡（爆炸或消失），这就是 DeepSeek-V4 引入 “流形约束 (mHC)” 来修正 Bl 的原因。

3. mHC就是在HC的基础上，将B变为双随机矩阵

双随机矩阵：数学定义

一个 $n \times n$ 的矩阵 A 如果被称为双随机矩阵，它必须同时满足以下三个条件：

约束条件	数学表达式	含义
非负性	$a_{ij} \geq 0$	矩阵里的所有元素都大于等于 0
行归一化	$\sum_j a_{ij} = 1$	每一行的元素之和等于 1
列归一化	$\sum_i a_{ij} = 1$	每一列的元素之和等于 1

核心性质（为什么 DeepSeek 要用它？）

在 DeepSeek 的 mHC 架构中，双随机矩阵之所以被选中，是因为它拥有三个完美的数学性质，恰好解决了深层网络训练的痛点：

 性质一：谱范数有界（防止爆炸）

原理：双随机矩阵的谱范数（最大奇异值）严格等于 **1**。

作用：这意味着当你用这个矩阵去乘一个向量（信号）时，**信号的幅度（范数）不会被放大**。

解决痛点：在深层网络中，如果矩阵的谱范数大于 1，信号经过几十层连乘后会指数级爆炸（梯度爆炸）；如果小于 1，信号会消失。双随机矩阵像一个 “**非扩张算子**”，保证了信号在传输过程中是安全的、有界的。

 性质二：乘法封闭性（深层稳定）

原理：两个双随机矩阵相乘，结果**依然是**双随机矩阵。

作用：这意味着无论你的网络堆叠多少层（比如 100 层），整个链路的复合映射依然保持 “谱范数为 1” 和 “行列和为 1” 的特性。

解决痛点：保证了超深层网络在训练时的数值稳定性，不会因为层数加深而导致系统崩溃。

 性质三：凸组合与特征融合（信息混合）

原理：由于元素非负且归一化，矩阵乘法本质上是在做**加权平均**（凸组合）。

作用：它不会凭空创造信息，也不会消灭信息，而是对特征进行**平滑和混合**。

解决痛点：在 mHC 中，这相当于一种稳健的特征融合机制，让不同通道的信息能够安全地交互。

如何生成？（Sinkhorn-Knopp 算法）

在代码实现中，我们很难直接学出一个满足行列和都为 1 的矩阵。DeepSeek 使用了 **Sinkhorn-Knopp 算法** 来“强行”把任意矩阵变成双随机矩阵：

取指数：先把矩阵里的数都变成正数（ $\exp$ ）。

交替归一化：

先把每一行除以行和（让行和变 1）。

再把每一列除以列和（让列和变 1）。

重复上述步骤几次（比如 20 次），矩阵就会快速收敛，同时满足行和与列和都为 1。

总结：为什么要用它？

维度	自由矩阵 (HC)	双随机矩阵 (mHC)	带来的好处
信号幅度	无限放大 (3000x)	严格限制 ($\leq 1x$)	训练不崩，Loss 收敛
深层叠加	越乘越乱	越乘越稳	支持万亿参数大模型
信息流动	可能丢失或畸变	守恒流动	特征提取更精准

所以，DeepSeek 使用双随机矩阵，本质上是用数学约束换取了工程上的极致稳定性，让“宽通道”（HC）这种高性能架构真正变得可用。

二、Hybrid Attention with CSA and HCA (注意力，大幅提升长上下文效率)

CSA：压缩稀疏注意力机制

CSA 通过两件事来降低注意力机制的开销。

第一步：压缩。CSA 不是为每个 token 都维护一个键值缓存，而是使用学习到的 softmax 权重将每 m 个连续的 token 合并成一个压缩条目。

为了便于理解，假设 $m = 4$ 。那么每 4 个 token 就会合并成一个键值缓存条目。键值缓存的大小缩小了 m 倍。

在我们的图书馆示例中，这就像将每 4 页内容合并成一张摘要卡片。这样，我们不再需要阅读 100 万页内容，而是只需查看 25 万张摘要卡片。

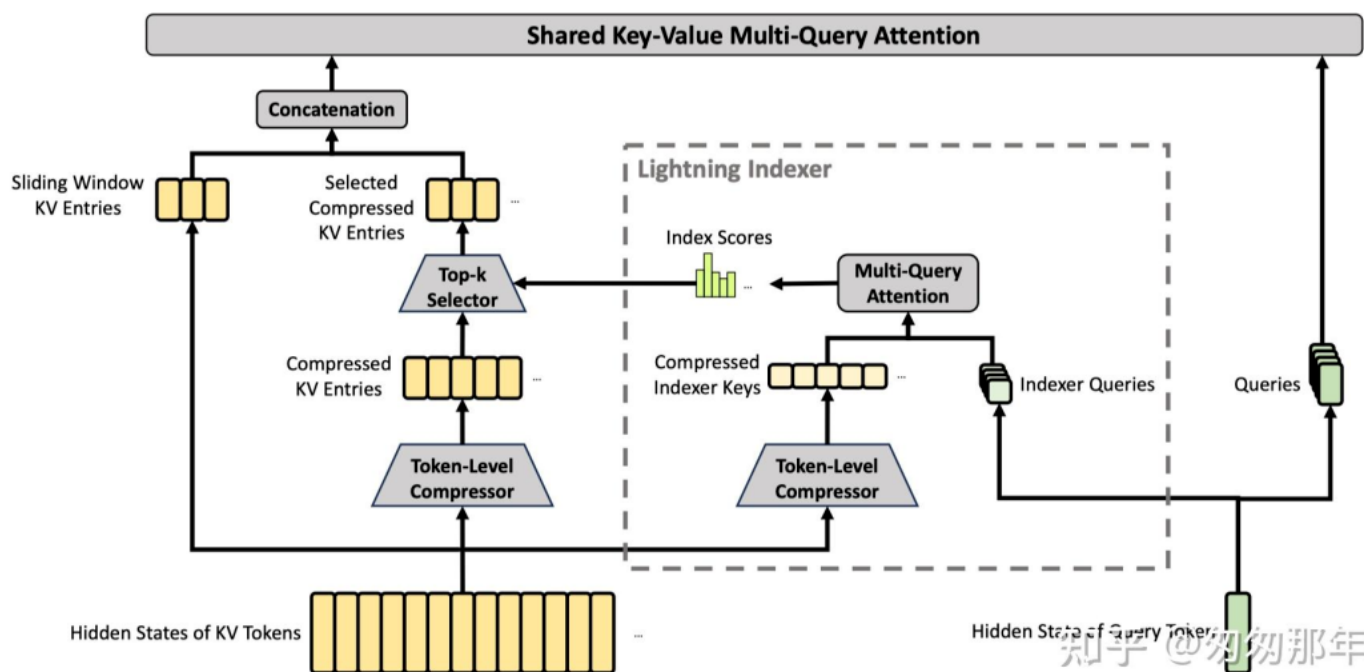
步骤 2：选择。压缩后，CSA 仍然不会查看每一张摘要卡片。它使用一个小型 Lightning Indexer 对每条压缩后的条目与当前查询进行评分，然后只选择相关性最高的前 k 条。

Lightning Indexer 以 FP4 精度运行，以降低评分步骤的成本。

```

1  All tokens:  [t1][t2][t3][t4][t5][t6][t7][t8] ...
2              ||  ||
3              group 1      group 2
4              |            |
5  Compressed:  [C1]        [C2]        [C3] ...
6              |            |            |
7  Lightning Indexer scores each one against current query
8              |            |            |
9              score=0.9    score=0.2    score=0.8
10             |            |            |
11 Top-k selection: [ C1 ]    ( C2 )    [ C3 ]
12                 picked    skipped    picked
13                 |            |
14             +-----+-----+
15                 |
16                 v
17             Attention runs only on [C1] and [C3]
18                 (the picked compressed entries)

```



因此，CSA 首先将多个词元压缩成一个，然后只选择其中看起来最相关的少数压缩条目。这就是 CSA 降低成本的方式。计算和内存都得到了节省。

举例说明：

[未命名文档](#)

HCA：重压缩注意力

HCA 采取了不同的权衡策略。它的压缩率更高，而且由于压缩后的序列已经很小，因此对其进行密集注意力的成本很低。无需稀疏选择。

HCA 使用的分组大小 m' 比 CSA 中的 m 大得多。对于 DeepSeek-V4， m' 设置为 128。

因此，HCA 不是将 4 个词元合并成一个条目，而是将 128 个词元合并成一个条目。键值缓存的缩减幅度更大。

在我们的图书馆示例中，这相当于将每 128 页的内容分组成一张摘要卡片。

现在我们总共只有大约 7800 张摘要卡片。由于卡片数量已经非常少，我们每次都读取所有卡片，无需选择前几张。

因此，CSA 和 HCA 是两种不同的设计思路，但目标相同——低成本的长上下文注意力机制。

CSA 采用轻度压缩加上稀疏的 top-k 选择，而 HCA 采用激进压缩加上密集注意力机制。两者的最终结果都是：计算量和内存占用都非常低。

以上是关于 HCA 的全部内容。现在，让我们看看实际模型是如何同时使用这两种机制的。

在 DeepSeek-V4 中，各层是交错排列的。对于 DeepSeek-V4-Flash，前两层使用纯滑动窗口注意力机制，其余层则交替使用 CSA 和 HCA。

对于 DeepSeek-V4-Pro，前两层使用 HCA，其余层则交替使用 CSA 和 HCA。

这样，不同的层以略微不同的方式处理长上下文，共同实现了细粒度选择（CSA）和广度覆盖（HCA）。

三、Muon Optimizer（优化器，更快的收敛速度和更大的训练稳定性）

Muon 是一种使用完整梯度矩阵并对更新方向进行正交化的优化器。

这有什么意义呢？大多数优化器（例如 AdamW）会**独立处理**每个权重。而 Muon 则会**查看完整**的梯度矩阵，并在进行权重更新之前应用正交化步骤。

这种正交化是通过**牛顿-舒尔茨**迭代实现的。它将更新矩阵的所有奇异值归一化到接近 1，从而避免更新过程中出现单一方向的主导作用。最终结果是更快的收敛速度和更稳定的训练。

通常， M 会首先被归一化为

$$M_0 = \frac{M}{\|M\|_F}$$

以确保其最大奇异值不超过 1。随后，每一次 Newton-Schulz 迭代执行以下运算：

$$M_k = aM_{k-1} + b(M_{k-1}M_{k-1}^T - I)M_{k-1} + c(M_{k-1}M_{k-1}^T - I)^2M_{k-1}. \quad (28)$$

混合 Newton-Schulz 方法在两个不同阶段共执行 10 次迭代。在前 8 步中，我们使用系数 $(a,b,c)=(3.4445, -4.7750, 2.0315)$ 来驱动快速收敛，使奇异值迅速逼近 1。在最后的 2 步中，我们切换为系数

(a,b,c)=(2,-1.5,0.5)，从而将奇异值精确且稳定地控制在 1。经过这 10 步“精修”后，原本歪斜的矩阵 M 就变成了一个方向极其规整的正交矩阵。优化器再用这个“规整”后的方向去更新模型的万亿个参数，整个训练过程就会像火车在笔直的铁轨上行驶一样，又快又稳，完全不会脱轨（Loss Spike）。

Algorithm 1 Muon Optimizer for DeepSeek-V4

Require: Learning rate η , momentum μ , weight decay λ , update rescaling factor γ

1: **for** each training step t **do**

2: **for** each logically independent weight $W \in \mathbb{R}^{n \times m}$ **do**

3: $G_t = \nabla_W \mathcal{L}_t(W_{t-1})$ ▷ Compute gradients

4: $M_t = \mu M_{t-1} + G_t$ ▷ Accumulate momentum buffer

5: $O'_t = \text{HybridNewtonSchulz}(\mu M_t + G_t)$ ▷ Nesterov trick and hybrid Newton-Schulz

6: $O_t = O'_t \cdot \sqrt{\max(n, m) \cdot \gamma}$ ▷ Rescale the update RMS

7: $W_t = W_{t-1} \cdot (1 - \eta\lambda) - \eta O_t$ ▷ Perform weight decay and update

8: **end for**

9: **end for**

知乎 @匆匆那年

几个要点：

- AdamW 仍然用于嵌入模块、预测头、所有 RMSNorm 模块以及 mHC 的静态部分。
- Muon 用于其他所有部分。
- DeepSeek 团队采用了一种混合的牛顿-舒尔茨迭代法，并精心选择系数以实现稳定的正交化。

这让我们能够兼顾两者的优势。 μ 子迭代法在关键区域实现了更快的收敛速度，而 AdamW 迭代法则继续在已被证明行之有效的区域发挥作用。

四、FP4 Quantization-Aware Training

DeepSeek-V4 并没有简单粗暴地将所有参数一刀切地量化为 FP4，而是采用了一套极其精细的混合精度策略：

- 专家权重的极致压缩（FP4）：在 MoE（混合专家）架构中，模型将庞大的专家参数权重（Expert Weights）量化为 FP4。这使得 V4-Pro 的总参数量高达 1.6 万亿，但实际激活参数仅为 490 亿，极大地降低了模型加载和调用的显存门槛。
- 计算路径的无损转换（FP4 $\rightarrow$ FP8）：为了保证计算精度，DeepSeek-V4 引入了一个巧妙的转换机制。在推理计算时，系统会将 FP4 的子块（ 1×32 tiles）无损反量化为 FP8 参与计算。由于 FP8（E4M3）比 FP4（E2M1）多 2 个指数位，动态范围更大，能够完全吸收 FP4 子块内的精细尺度信息，从而在不增加存储压力的前提下，保证了计算过程的稳定性。

DeepSeek-V4 在两个关键位置应用了 FP4：

- MoE 专家权重，它占用了大部分 GPU 内存。
- CSA 索引器的查询键 (QK) 路径，闪电索引器在该路径中以 FP4 格式计算分数。

代码块 这里的“FP4 格式计算”指的是参与这个分数计算的输入数据（Q 和 K）以及中间运算过程是运行在极低精度下的：

- 2 输入压缩：索引器专用的查询向量（ $q_{Ih}(t)$ ）和压缩后的键向量（ k_{Is} ）在内存中是以 FP4（4-bit 浮点数）格式存储的。
- 3 计算过程：当计算点积（QK 乘法）时，硬件会直接读取这些 FP4 数据进行运算。为了极致效率，中间的 ReLU 激活等步骤通常也会配合低精度流水线进行。
- 4 结果精度：虽然输入和计算是 FP4，但为了保证后续 Top-K 筛选（比如比较 0.431 和 0.385 谁更大）的绝对准确，最终累加得到的 `score_s` 通常会转换回较高精度（如 FP32 或 BF16）来进行排序和决策。

五、Pre-Training: Anticipatory Routing and SwiGLU Clamping

DeepSeek-V4 团队通过这两项工程化极强的技巧，在不牺牲模型性能的前提下，有效解决了超大规模 MoE 模型训练中的“损失尖峰”难题：（“强迫均匀”的目标会和“让模型学得好”的主目标发生冲突）

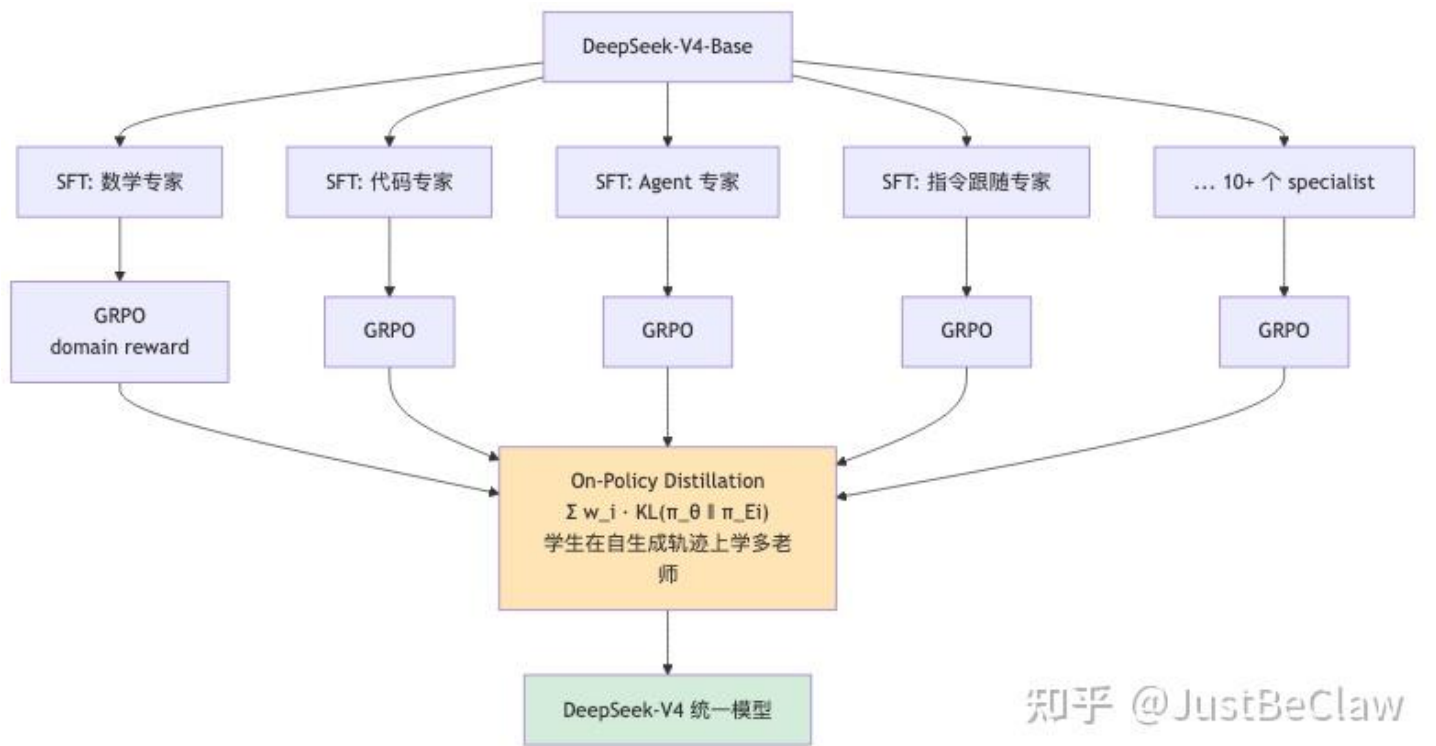
1. 前瞻路由（Anticipatory Routing）：打破恶性循环

- 核心原理：解耦骨干网络与路由网络的参数更新。计算路由时使用“过时”的历史参数，从而打破两者同步更新可能引发的误差放大恶性循环。
- 工程优化：
 - 提前计算：提前一步计算并缓存**路由索引**，避免重复加载参数。
(就是 MoE 模型里的“精准派单号”。它决定了每个字该由哪几个神经网络模块来处理，是让大模型在拥有万亿参数的同时，还能保持极高运算效率的核心调度指令。)
 - 通信掩盖：将预计算开销与专家并行（EP）的通信时间重叠，额外开销仅约 20%。
 - 动态触发：平时正常训练，仅在检测到损失尖峰时自动触发该机制，将整体训练开销降至几乎为零。

2. SwiGLU 截断（SwiGLU Clamping）：暴力抑制异常值

- 核心原理：直接对激活函数的输出数值范围进行强制限制，从源头上压制导致训练不稳定的异常值。
- 具体操作：
 - SwiGLU 线性部分：限制在 `[-10, 10]`。
 - SwiGLU 门控部分：上限限制为 `10`。
 - SwiGLU 就位于这些被激活的专家网络内部

六、Post-Training: Specialist Training and On-Policy Distillation



🧠 第一阶段：专家训练（分化）

这一阶段的目标是培养各个领域的“偏科高手”。

- 独立训练：针对数学、代码、智能体、指令遵循等不同领域，分别独立训练专家模型。
- SFT 打底：先在高质量领域数据上进行监督微调（SFT），建立基础能力。
- RL 拔高：利用 GRPO（群体相对策略优化）算法，结合特定领域的奖励信号进行强化学习，使模型在各自领域达到最优水平。

💛 第二阶段：策略内蒸馏（统一）

这一阶段的目标是将所有专家的能力“合成”进一个统一的学生模型中。

- 学生自己写草稿（On-Policy 在线策略）：
在常规的蒸馏中，学生是模仿老师写好的范文。但在 OPD 中，学生模型（最终模型）必须自己先开口说话，生成自己的回答轨迹（草稿）。这保证了学生是在学习处理“自己真实会遇到的情况”，而不是老师完美的理想情况，消除了训练和真实推理之间的差距。
- 专家老师打分（全词表蒸馏）：
当学生写出一个词（Token）时，背后的数学专家、代码专家等“教师天团”会同时给出意见。它们不是简单打个分，而是提供一个完整的词表概率分布（即：在这个语境下，下一个词选 A 的概率是多少，选 B 的概率是多少）。

- **针对性加权：**
如果当前是数学题，数学专家的意见权重就最高；如果是写代码，代码专家的权重就最高。学生模型通过反向 KL 散度（Reverse KL Loss）来调整自己，让自己的概率分布尽可能向加权后的专家意见靠拢。

OPD 使用的“反向 KL 散度”是一种“模式搜索（Mode-seeking）”方法。通俗地说，如果数学专家说“答案绝对是 42”，代码专家说“这里应该用 print”，反向 KL 会强迫学生模型坚定地选择 42 和 print，而不是取平均值搞出一个模棱两可的废答案。这使得模型输出更加精准、清晰。

这一步骤**取代了 DeepSeek-V3.2 中使用的传统混合强化学习阶段**。团队发现，与权重合并或混合强化学习相比，OPD 能够提供更稳定、更可靠的知识迁移，因此它成为了 DeepSeek-V4 训练后流程的核心巩固步骤。

七、Reasoning Modes

推理模式	核心机制	典型适用场景	推荐上下文窗口下限
Non-think	旁路思维链，直读权重参数	日常问答、前端交互、文本概括	无特殊要求
Think High	激活标准思维链，适度逻辑反思	企业文档解析、代码调试、系统重构	> 10,000 Tokens
Think Max	深度强化学习搜索树，逻辑纠错	科研数学证明、高难度算法、多重代理规划	> 384,000 Tokens